Day 3

Filtering with WHERE: comparison operators, AND / OR / NOT

Phase	Time	You need
Phase 1: Foundations	60-90 minutes	Day 1 dataset loaded; any SQL editor (pgAdmin Query Tool or DB Browser for SQLite)

Learning objectives

- Use **WHERE** to return only the rows you care about.
- Compare numbers, text, and dates with **=, <>, <, >, <=, >=**.
- Combine conditions with **AND**, **OR**, and **NOT**.
- Use parentheses to control evaluation order and avoid the classic AND/OR bug.

Dataset reminder

Same four tables as Day 1: `customers`, `products`, `orders`, `order_items`. Today you mostly query one table at a time. Everything here runs on both PostgreSQL and SQLite; differences are flagged.

```
customers(customer_id, name, email, city, signup_date)
products(product_id, name, category, price)
orders(order_id, customer_id, order_date, status)
order_items(order_id, product_id, quantity, unit_price)
```

Before you start: I don't hard-code specific city, category, or status values in the exercises, because you should look at your own data first. That is also a quick Day 2 refresher: run `SELECT DISTINCT city FROM customers;` and see what you have. Wherever an exercise says *pick a value*, use one you actually saw.

1. Concept (15 min)

So far, `SELECT ... FROM ...` returns **every row** in a table. Real questions are narrower: *which* customers live in one city, *which* products cost more than a number. The `WHERE` clause is a filter. Each row is tested against your condition; rows where it is true are kept, the rest are dropped.

```
SELECT column_list
FROM table_name
WHERE condition;
```

Where WHERE goes

It always comes **right after FROM**. Think of the order as: pick the table, filter the rows, then show the columns.

Comparison operators

Operator	Meaning	Example
=	equal to (a single equals sign in SQL)	`city = 'Pune'`

Operator	Meaning	Example
<> or !=	not equal to	`status <> 'cancelled'`
>	greater than	`price > 100`
<	less than	`price < 100`
>=	greater than or equal to	`quantity >= 2`
<=	less than or equal to	`unit_price <= 50`

Writing values correctly

- **Numbers** have no quotes: `price > 100`.
- **Text** goes in **single quotes**: `city = 'Pune'`. Double quotes mean a column name in PostgreSQL, so `city = "Pune"` fails.
- **Dates** are written as text in `YYYY-MM-DD` form: `order_date >= '2025-01-01'`.
- **Case:** in PostgreSQL, text comparison is case-sensitive ('pune' is not 'Pune'). In SQLite, `=` is also case-sensitive by default. Match the case you see in the data.

AND, OR, NOT

A single condition is rarely enough. Combine them:

Keyword	Keeps a row when...	Example
`AND`	**both** conditions are true	`price > 20 AND category = 'Books'`
`OR`	**at least one** condition is true	`city = 'Pune' OR city = 'Delhi'`
`NOT`	the condition is **false** (flips it)	`NOT city = 'Pune'`

Order of evaluation (important)

SQL evaluates **NOT first, then AND, then OR**. That means `A OR B AND C` is read as `A OR (B AND C)`, which is often not what you meant. **Use parentheses** whenever you mix AND with OR; they make your intent explicit and are never wrong.

```
-- Intent: Books or Toys, but only those under 30
-- WRONG (AND binds tighter, so price < 30 only applies to Toys):
WHERE category = 'Books' OR category = 'Toys' AND price < 30

-- RIGHT:
WHERE (category = 'Books' OR category = 'Toys') AND price < 30
```

Sneak peek, full lesson tomorrow: a missing value (NULL) is not equal to anything, not even another NULL. Customer 7 has a NULL email, so `WHERE email = 'x'` and `WHERE email <> 'x'` both skip that row. Writing `email = NULL` never works. Day 4 gives you `IS NULL`.

2. Guided Practice (20 min)

Type each query yourself (do not copy-paste; typing builds muscle memory), run it, and read the result before moving on. Replace anything in **[brackets]** with a real value from your data.

Step 1: Look before you filter

```
SELECT * FROM customers;
SELECT DISTINCT city FROM customers;
SELECT DISTINCT status FROM orders;
SELECT DISTINCT category FROM products;
```

Write down the cities, statuses, and categories you see. You will reuse them.

Step 2: One condition

```
-- Text equality: use a city from your DISTINCT list
SELECT name, city FROM customers
WHERE city = '[a city from your data]';

-- Numeric comparison
SELECT name, price FROM products
WHERE price > 50;
```

Now change > to >= and to <. Compare the row counts. Which rows appear or disappear when a price is exactly 50?

Step 3: Not equal

```
SELECT order_id, status FROM orders
WHERE status <> '[a status from your data]';
```

Step 4: Filtering on a date

```
SELECT order_id, order_date FROM orders
WHERE order_date >= '2025-01-01';
```

Look at your real `order_date` values from Step 1 and adjust the date so you keep some rows but not all. (Day 6 and Day 21 go deeper on date functions.)

Step 5: AND

```
SELECT order_id, product_id, quantity, unit_price
FROM order_items
WHERE quantity >= 2 AND unit_price > 20;
```

Both conditions must hold. Try changing AND to OR and note how the row count changes. **Rule of thumb:** AND can only shrink your result; OR can only grow it.

Step 6: OR and NOT

```
SELECT name, city FROM customers
WHERE city = '[city A]' OR city = '[city B]';

SELECT name, city FROM customers
WHERE NOT city = '[city A]';
```

Step 7: The parentheses experiment

```
-- Query 1
SELECT name, category, price FROM products
WHERE category = '[cat A]' OR category = '[cat B]' AND price < 30;

-- Query 2
SELECT name, category, price FROM products
WHERE (category = '[cat A]' OR category = '[cat B]') AND price < 30;
```

Run both. If the row counts differ, you just saw the precedence rule in action. If they match on your data, try a price threshold that separates your products better (for example, a value between your cheapest and priciest).

3. Exercises (25-40 min)

Seven problems, easy to hard. Write the query first, then check the answer key. Your row counts depend on your data, so the key explains *why* each query is correct instead of printing results.

Exercise 1 [Easy]

List the `name` and `email` of every customer who lives in one city of your choice.

Exercise 2 [Easy]

List the `name` and `price` of all products that cost more than 25.

Exercise 3 [Easy]

List all orders whose status is **not** a particular status of your choice. Do it twice: once with <> and once with `NOT`. Confirm both give the same rows.

Exercise 4 [Medium]

From `order_items`, find the rows where `quantity` is at least 2 **and** `unit_price` is at most 50. Show `order_id`, `product_id`, `quantity`, `unit_price`.

Exercise 5 [Medium]

List the customers who live in either of two cities of your choice. Then list the customers who live in neither.

Exercise 6 [Hard]

List products that are in one of two categories of your choice **and** cost less than a threshold that you pick so the result is neither empty nor everything. Write it with parentheses, then rewrite it without and show how the result changes.

Exercise 7 [Hard]

Fix the broken queries (each has one bug; there are four):
(a) `SELECT * FROM customers WHERE city == 'Pune';`
(b) `SELECT * FROM customers WHERE city = Pune;`
(c) `SELECT * FROM products WHERE price > '50' AND OR category = 'Books';`
(d) `SELECT * FROM customers WHERE city = 'Pune' AND city = 'Delhi';` (it runs but returns nothing; the writer wanted customers from either city)

Stretch goal

Write one query on `orders` that returns orders placed in a date window you choose (use >= and <, so the end date is excluded) **and** whose status is not one of your two least interesting statuses, using `NOT` and parentheses. Then explain in one sentence why < with the day after is safer than <= when a date column might also store a time.

4. Recap (5 min)

One-line takeaway: `WHERE` keeps only the rows that make your condition true; combine conditions with AND / OR / NOT, and always use parentheses when you mix AND with OR.

Common mistakes

- **Using == instead of =.** SQL uses a single equals sign for comparison.
- **Forgetting quotes on text** (`city = Pune`) or **adding quotes to numbers** (`price > '50'`).
- **Mixing AND and OR without parentheses.** It runs without an error and quietly returns the wrong rows, which is the worst kind of bug.
- **Writing two values for one column with AND** (`city = 'A' AND city = 'B'`). One cell cannot hold two values; you want OR.
- **Case and spelling mismatches.** 'pune' is not 'Pune'. Check with `SELECT DISTINCT` first.
- **Comparing to NULL with = or <>.** It silently skips those rows. Coming on Day 4.

Tomorrow (Day 4)

`IN`, `BETWEEN`, `LIKE`, and NULL handling: shorter, cleaner ways to write the filters you wrote today.

5. Answer Key with Explanations

Replace the bracketed values with your own. Each explanation is the reasoning to compare against your attempt.

Exercise 1

```
SELECT name, email
FROM customers
WHERE city = '[your city]';
```

Text needs single quotes and an exact-case match. Because customer 7 has a NULL email, that customer can still appear in the result if they live in your city; the NULL only matters when you *filter on* email.

Exercise 2

```
SELECT name, price
FROM products
WHERE price > 25;
```

A plain numeric comparison with no quotes. A product costing exactly 25 is excluded; use >= if you wanted it included.

Exercise 3

```
SELECT * FROM orders WHERE status <> '[status]';
SELECT * FROM orders WHERE NOT status = '[status]';
```

Both flip the test and return the same rows. <> is usually more readable for a single comparison; NOT shines in front of bigger conditions (for example, `NOT (a AND b)`).

Exercise 4

```
SELECT order_id, product_id, quantity, unit_price
FROM order_items
WHERE quantity >= 2 AND unit_price <= 50;
```

AND means a row must pass **both** tests. The boundary operators matter: "at least" is >=, "at most" is <=. If your result is empty, your data has no row meeting both; loosen one number and re-run.

Exercise 5

```
SELECT name, city FROM customers
WHERE city = '[A]' OR city = '[B]';

SELECT name, city FROM customers
WHERE city <> '[A]' AND city <> '[B]';
```

"Either" is OR. "Neither" is the opposite, and the classic trap is writing *OR* in the second query: `city <> 'A' OR city <> 'B'` is true for every row (a city can't equal both at once), so nothing is filtered. "Not A **and** not B" is correct. (This is De Morgan's law: `NOT (A OR B)` equals `NOT A AND NOT B`.)

Exercise 6

```
SELECT name, category, price
FROM products
WHERE (category = '[A]' OR category = '[B]')
  AND price < [threshold];

-- Without parentheses: AND is evaluated first, so this means
-- category = A  OR  (category = B AND price < threshold)
SELECT name, category, price
FROM products
WHERE category = '[A]' OR category = '[B]' AND price < [threshold];
```

With parentheses, the price limit applies to both categories. Without them, **every** product in category A is returned regardless of price, because only the B branch is combined with the price test. The result usually has more rows than intended.

Exercise 7

```
-- (a) Use a single =
SELECT * FROM customers WHERE city = 'Pune';
-- (b) Text needs quotes (otherwise Pune is read as a column name)
SELECT * FROM customers WHERE city = 'Pune';
-- (c) Remove the stray AND and the quotes around the number
SELECT * FROM products WHERE price > 50 OR category = 'Books';
-- (d) One cell cannot be two cities: use OR
SELECT * FROM customers WHERE city = 'Pune' OR city = 'Delhi';
```

(a) == is a syntax error in PostgreSQL. (b) Without quotes the database looks for a column named Pune and errors out. (c) `AND OR` is invalid; and while quoting a number sometimes works because of automatic type conversion, it is a bad habit and in PostgreSQL can raise an error or compare as text. Which of AND or OR you want depends on the question; either way, the keyword must stand alone. (d) Valid SQL, wrong logic: contradictory conditions return zero rows without any error message.

Stretch goal: model answer

```
SELECT order_id, order_date, status
FROM orders
WHERE order_date >= '[start date]'
  AND order_date <  '[day after end date]'
  AND NOT (status = '[boring 1]' OR status = '[boring 2]');
```

Using < with the **next day** is safer than <= with the last day because a timestamp like '2025-03-31 14:30' is greater than '2025-03-31', so `<= '2025-03-31'` would silently drop all of that day's later rows. The parentheses around the two statuses are needed so NOT applies to both.